

Self-Supervised Learning for Time Series

Representations, TS2Vec, T-Rep, and newer directions

Mikolaj Slupinski

May 2026

After this lecture you should be able to:

- Explain what a time-series representation should preserve.
- Translate contrastive and masked pretext tasks to temporal data.
- Describe TS2Vec's contextual consistency and hierarchical contrast.
- Describe how T-Rep adds learned time embeddings and new pretext tasks.
- Choose a representation strategy for forecasting, classification, anomaly detection, or missing data.

1. Why self-supervised learning for time series?
2. Short reminder: contrastive and masked objectives
3. TS2Vec in detail
4. T-Rep in detail
5. Newer approaches and future directions
6. Practical takeaways

Motivation

Why self-supervised learning?

Time series are often abundant, but labels are scarce.

- Sensors, logs, financial streams, and medical monitors generate many unlabeled windows.
- Labels may require domain experts or delayed outcomes.
- Downstream tasks often reuse the same raw measurements.

Goal: pretrain an encoder on unlabeled sequences, then reuse its representations with small supervised heads.

What is a time-series representation?

For a multivariate sequence

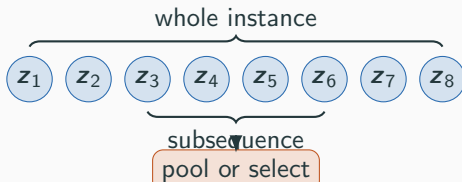
$$\mathbf{X}_i = [\mathbf{x}_{i,1}, \dots, \mathbf{x}_{i,T}] \in \mathbb{R}^{T \times C},$$

an encoder returns

$$f_{\theta}(\mathbf{X}_i) = \mathbf{Z}_i = [\mathbf{z}_{i,1}, \dots, \mathbf{z}_{i,T}] \in \mathbb{R}^{T \times K}.$$

- $\mathbf{z}_{i,t}$ describes a timestamp in context.
- Pooling over t gives an instance-level representation.
- Pooling over a window gives a subsequence representation.

Representation granularity



classification uses an instance vector;

forecasting and anomaly detection often need timestamp vectors

Downstream tasks reuse the same encoder

Task	Representation used	Typical head
Classification	pooled sequence	SVM, MLP, linear probe
Forecasting	last or window representation	ridge regression, MLP
Anomaly detection	timestamp representation	score or classifier
Imputation	local context representation	reconstruction head

A useful pretrained encoder should support several heads without changing the representation backbone.

Why time series are different

- Magnitude can be semantic: scaling a signal may change its meaning.
- Cropping can change distribution because the process may be non-stationary.
- Nearby points can be very different during spikes or changepoints.
- Far points can be similar because of periodicity.
- Missingness may be structured, not random noise.

The main design question is: **which invariances are safe?**

A leakage reminder

Self-supervised pretraining does not remove the usual time-series risks.

- Do not pretrain on test-period data if deployment would not have seen it.
- Fit scalers and imputers on the training period only.
- Validate using chronological splits when the application is forecasting.
- Treat calendar covariates as deployment assumptions.

The encoder can exploit leakage even when the downstream head is simple.

SSL Reminder

Pretext task vs downstream task

Pretext task: an artificial supervised problem created from unlabeled data.

unlabeled series $\rightarrow$ views, masks, targets, positives, negatives

Downstream task: the real task evaluated after pretraining.

$$\mathbf{z} = f_{\theta}(\mathbf{X}), \quad \hat{y} = g_{\phi}(\mathbf{z}).$$

The pretext task is useful only if it forces the encoder to preserve information needed later.

Contrastive learning in one slide

Contrastive learning builds pairs:

- **positive:** two views that should encode similarly,
- **negative:** examples that should be distinguishable.

For anchor $\mathbf{z}$, positive $\mathbf{z}^+$, and negatives $\{\mathbf{z}_j^-\}$:

$$\ell = -\log \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau)}{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau) + \sum_j \exp(\text{sim}(\mathbf{z}, \mathbf{z}_j^-)/\tau)}.$$

The hard part is choosing positives and negatives that match the data.

What is a positive pair in time series?

Strategy	Assumption	Risk
Two augmentations	augmentation preserves meaning	false invariance
Nearby windows	local smoothness	misses spikes
Same instance/subseries	same process	level shifts
Same timestamp in context	local value is preserved	context choice matters

TS2Vec is mainly about choosing positives more carefully for time series.

Masked modeling reminder

Masked modeling hides part of the input and asks the model to reconstruct or predict it.

$$\tilde{\mathbf{X}} = \text{mask}(\mathbf{X}), \quad \hat{\mathbf{X}}_{\mathcal{M}} = g_{\phi}(f_{\theta}(\tilde{\mathbf{X}})).$$

- In images, patches have bounded pixel ranges.
- In language, a special mask token is natural.
- In time series, raw values may be unbounded and zero may be a real value.

Masked autoencoders for time series

A time-series masked autoencoder usually learns from:

$$\mathcal{L}_{rec} = \sum_{t \in \mathcal{M}} \|\hat{\mathbf{x}}_t - \mathbf{x}_t\|^2.$$

Useful when:

- local context is informative,
- missing values are important at deployment,
- reconstruction forces the model to learn shape and seasonality.

But: reconstruction alone may overemphasize low-level details.

Downstream heads should be simple

The representation is the main object of study.

- Classification: train an SVM or linear probe on pooled representations.
- Forecasting: fit a ridge regression or small MLP on timestamp representations.
- Anomaly detection: compare normal-context and masked-context representations.

If a very large downstream model is needed, the representation quality is harder to interpret.

What should SSL preserve?

- **Shape:** local motifs, spikes, oscillations.
- **Level:** absolute magnitude when it matters.
- **Time:** ordering, seasonality, phase, trend.
- **Context:** what happened before and after.
- **Identity:** differences between instances or systems.
- **Uncertainty:** enough information to support probabilistic decisions.

Evaluation checklist

- Does the pretraining split respect time?
- Are the downstream heads simple and comparable?
- Are results reported across tasks, not only classification?
- Are missing-data and irregular-sampling regimes tested?
- Are ablations tied to the proposed pretext tasks?

For general representations, single-task accuracy is not enough.

TS2Vec

TS2Vec: the central idea

TS2Vec learns timestamp-level representations by contrasting two augmented context views.

Three core ideas:

- positives are the same timestamp under different contexts,
- contrast happens across time and across instances,
- the loss is applied at multiple temporal resolutions.

Main source: Yue et al., TS2Vec, arXiv:2106.10466.

TS2Vec problem definition

For N series

$$\mathcal{X} = \{\mathbf{X}_1, \dots, \mathbf{X}_N\}, \quad \mathbf{X}_i \in \mathbb{R}^{T \times C},$$

learn

$$f_{\theta}(\mathbf{X}_i) = \mathbf{R}_i = [\mathbf{r}_{i,1}, \dots, \mathbf{r}_{i,T}], \quad \mathbf{r}_{i,t} \in \mathbb{R}^K.$$

- Each timestamp gets its own contextual representation.
- A subsequence can be represented by pooling over its timestamps.
- The whole instance is represented by global pooling.

TS2Vec architecture

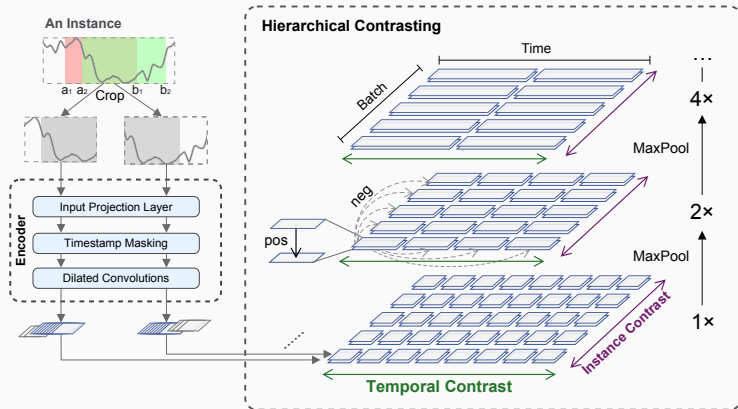


Figure source: Yue et al., TS2Vec, arXiv:2106.10466.

Encoder components

1. **Input projection:** map each observation $\mathbf{x}_{i,t}$ to a latent vector.
2. **Timestamp masking:** randomly set projected vectors to a mask value.
3. **Dilated CNN:** produce contextual timestamp representations.

$$\mathbf{x}_{i,t} \rightarrow \mathbf{z}_{i,t} \rightarrow \tilde{\mathbf{z}}_{i,t} \rightarrow \mathbf{r}_{i,t}.$$

The dilated CNN gives a large receptive field without recurrence.

Why mask after projection?

Raw time-series values do not have a natural universal mask token.

- A value of zero may be a real observation.
- The raw value range may be unbounded.
- A learned projection can make the zero vector a clear latent mask.

TS2Vec masks latent vectors after the input projection:

$$\tilde{\mathbf{z}}_{i,t} = m_t \mathbf{z}_{i,t}, \quad m_t \sim \text{Bernoulli}(p).$$

The TS2Vec supplement reports better UCR accuracy when masking after projection.

Contextual consistency

Sample two overlapping crops of the same series:

$$[a_1, b_1], \quad [a_2, b_2], \quad a_1 \leq a_2 \leq b_1 \leq b_2.$$

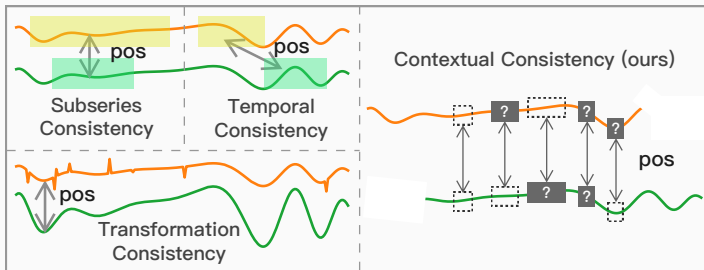
Only the overlap matters for the contrastive objective:

$$\Omega = [a_2, b_1].$$

Positive pair: representations of the same timestamp $t \in \Omega$ under two different contexts.

The representation should describe the timestamp, not memorize the crop boundary.

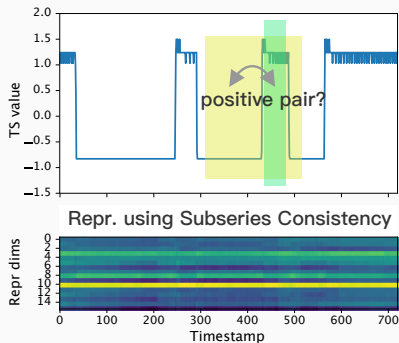
Positive pair selection strategies



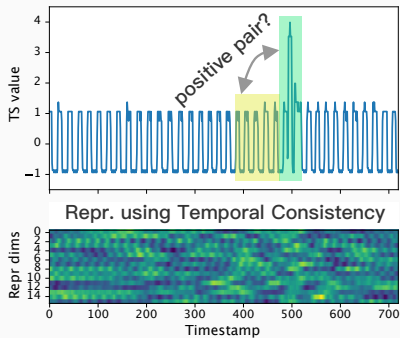
TS2Vec chooses same-timestamp positives from different augmented contexts.

Figure source: Yue et al., TS2Vec, arXiv:2106.10466.

Why older positives can fail



Level shifts



Anomalies

Nearby or same-subseries positives can force different regimes to look similar.

Figure source: Yue et al., TS2Vec, arXiv:2106.10466.

Two augmentations used by TS2Vec

Timestamp masking

- hides projected timestamp vectors,
- encourages inference from surrounding context,
- is applied independently in each forward pass.

Random cropping

- gives two overlapping context windows,
- prevents the model from solving contrast by absolute position,
- helps avoid representation collapse.

Temporal contrastive loss

For one instance i and timestamp t , the positive is the same timestamp in the other view:

$$(\mathbf{r}_{i,t}, \mathbf{r}'_{i,t}).$$

Negatives are other timestamps in the overlap:

$$\ell_{temp}^{(i,t)} = -\log \frac{\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t})}{\sum_{t' \in \Omega} \left[\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t'}) + \mathbb{1}_{t \neq t'} \exp(\mathbf{r}_{i,t} \cdot \mathbf{r}_{i,t'}) \right]}.$$

Instance-wise contrastive loss

At a fixed timestamp t , the positive is the same instance in the other view:

$$(\mathbf{r}_{i,t}, \mathbf{r}'_{i,t}).$$

Negatives are other instances in the batch:

$$\ell_{inst}^{(i,t)} = -\log \frac{\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{i,t})}{\sum_{j=1}^B \left[\exp(\mathbf{r}_{i,t} \cdot \mathbf{r}'_{j,t}) + \mathbb{1}_{i \neq j} \exp(\mathbf{r}_{i,t} \cdot \mathbf{r}_{j,t}) \right]}.$$

The dual contrastive objective

TS2Vec combines temporal and instance-wise discrimination:

$$\mathcal{L}_{dual} = \frac{1}{BT} \sum_i \sum_t \left(\ell_{temp}^{(i,t)} + \ell_{inst}^{(i,t)} \right).$$

- Temporal contrast separates different moments within a series.
- Instance contrast separates different series at the same timestamp.
- Together, they encode dynamics and instance identity.

Algorithm 1: Calculating the hierarchical contrastive loss

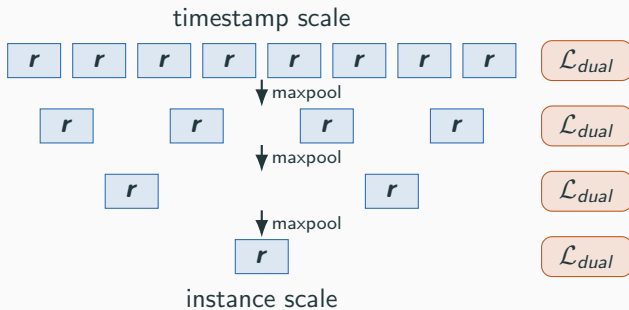
Input: two augmented representation tensors $R, R' \in \mathbb{R}^{B \times T \times K}$.

```
1   $\mathcal{L}_{hier} \leftarrow \mathcal{L}_{dual}(R, R')$ 
2   $d \leftarrow 1$ 
3  while time_length( $R$ ) > 1 do
4     $R \leftarrow \text{MaxPool1D}(R, \text{kernel} = 2)$ 
5     $R' \leftarrow \text{MaxPool1D}(R', \text{kernel} = 2)$ 
6     $\mathcal{L}_{hier} \leftarrow \mathcal{L}_{hier} + \mathcal{L}_{dual}(R, R')$ 
7     $d \leftarrow d + 1$ 
8  return  $\mathcal{L}_{hier} / d$ 
```

Each pooling step halves the temporal resolution, so the same contrastive objective is applied from timestamp scale to instance scale.

Algorithm adapted from Yue et al., TS2Vec, arXiv:2106.10466.

Hierarchical contrasting



Apply the same contrastive objective after repeatedly pooling along time.

Why hierarchy matters

Different downstream tasks need different semantic levels.

- Anomaly detection may need a representation at a single timestamp.
- Forecasting may need the latest contextual vector.
- Classification may need one vector for the whole instance.
- Subsequence retrieval may need a representation for a selected interval.

Hierarchical loss makes the encoder useful across these granularities.

Representing any subsequence

Because TS2Vec outputs $\mathbf{r}_{i,t}$ for every timestamp, a subsequence representation is simple:

$$\mathbf{r}_{i,a:b} = \text{pool}(\mathbf{r}_{i,a}, \dots, \mathbf{r}_{i,b}).$$

- Use global max pooling for instance-level classification.
- Use the last timestamp vector for forecasting.
- Use timestamp vectors directly for anomaly scoring.

$$\mathbf{r}_i^{global} = \max_{t=1,\dots,T} \mathbf{r}_{i,t}.$$

Then fit a standard classifier:

$$\hat{y}_i = g_\phi(\mathbf{r}_i^{global}).$$

In the TS2Vec evaluation, the classifier is an SVM with an RBF kernel.

Interpretation: the self-supervised encoder does the heavy lifting; the supervised layer is intentionally simple.

For a forecast at time t , encode the history and use the contextual vector at the last timestamp:

$$\mathbf{r}_t = f_{\theta}(\mathbf{x}_{1:t})_t.$$

A linear model predicts future values:

$$\hat{\mathbf{y}}_{t+1:t+H} = W\mathbf{r}_t + \mathbf{b}.$$

This tests whether the representation already contains predictive information.

Compare two representations of the same timestamp:

- unmasked input representation $\mathbf{r}_t^u$,
- masked input representation $\mathbf{r}_t^m$.

An anomaly score can be based on their distance:

$$\alpha_t = \|\mathbf{r}_t^u - \mathbf{r}_t^m\|_1.$$

If the observed point is unusual, the masked context predicts a different representation than the unmasked one.

TS2Vec classification results

Method	125 UCR datasets			29 UEA datasets		
	Avg. Acc.	Avg. Rank	Time (h)	Avg. Acc.	Avg. Rank	Time (h)
TNC	0.761	3.52	228.4	0.677	3.84	91.2
TS-TCC	0.757	3.38	1.1	0.682	3.53	3.6
T-Loss	0.806	2.73	38.0	0.675	3.12	15.1
TS2Vec	0.830	1.82	0.9	0.712	2.40	0.6

TS2Vec improves average accuracy while being much faster than several contrastive baselines.

Table recreated from Yue et al., TS2Vec, arXiv:2106.10466.

TS2Vec forecasting summary

Dataset	ETTh ₁	ETTh ₂	ETTm ₁	Electric.	Avg.
TS2Vec	0.110	0.170	0.069	0.486	0.209
Informer	0.186	0.204	0.241	0.607	0.310
LogTrans	0.196	0.217	0.270	0.795	0.370
TCN	0.263	0.219	0.200	0.673	0.338

The paper reports strong forecasting from a simple linear model trained on TS2Vec representations.

MSE averages recreated from the TS2Vec univariate forecasting table.

A forecasting slice

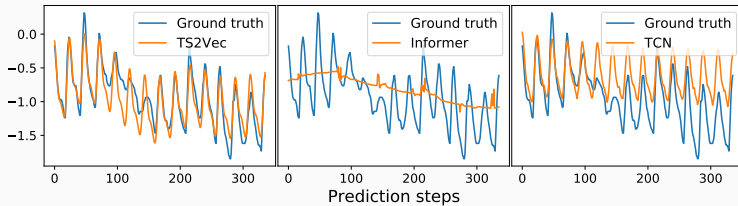
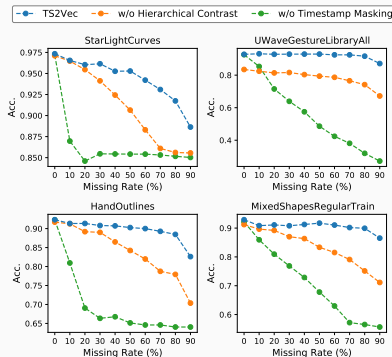


Figure source: Yue et al., TS2Vec, arXiv:2106.10466.

Robustness to missing data



Timestamp masking and hierarchical context help the encoder remain useful with missing points.

Figure source: Yue et al., TS2Vec, arXiv:2106.10466.

TS2Vec ablation summary

Variant	Avg. UCR accuracy
TS2Vec	0.829
w/o Temporal Contrast	0.819
w/o Instance Contrast	0.824
w/o Hierarchical Contrast	0.812
w/o Random Cropping	0.808
w/o Timestamp Masking	0.820
Contextual → Temporal Consistency	0.807
Contextual → Subseries Consistency	0.780
Dilated CNN → LSTM	0.779
Dilated CNN → Transformer	0.647

Table recreated from the TS2Vec ablation study.

- The main innovation is not only a contrastive loss; it is the construction of context views.
- Timestamp-level outputs make the method flexible across tasks.
- Hierarchical contrast connects fine and coarse temporal semantics.
- Random cropping and masking are not generic decoration; they prevent specific time-series failures.
- TS2Vec becomes a strong baseline for later time-series SSL models.

T-Rep

Why go beyond TS2Vec?

TS2Vec treats time mostly through contrastive identities:

- same timestamp: positive,
- different timestamp: negative.

That signal is binary.

T-Rep question: can the model learn a smoother and more explicit representation of time itself?

Main source: Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep in one sentence

T-Rep extends TS2Vec-style timestamp representations with learned time embeddings and two new pretext tasks.

signal representation + learned time representation
→ structured temporal latent space.

Target properties:

- smoother latent trajectories,
- better missing-data robustness,
- stronger forecasting and anomaly detection,
- more interpretable time-local representations.

T-Rep architecture

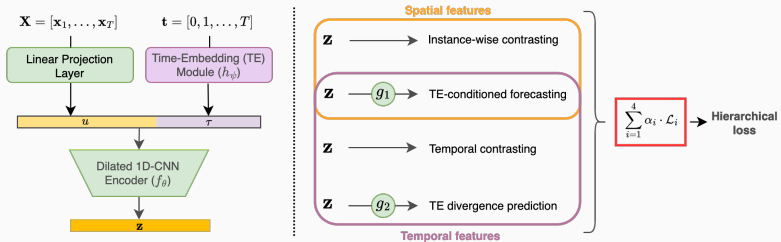


Figure source: Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep uses three main encoder components.

1. **Linear projection:** $\mathbf{x}_{i,t} \mapsto \mathbf{u}_{i,t}$.
2. **Time-embedding module:** $t \mapsto \boldsymbol{\tau}_t$.
3. **Dilated CNN encoder:** consumes $[\mathbf{u}_{i,t}; \boldsymbol{\tau}_t]$ and outputs $\mathbf{z}_{i,t}$.

$$\mathbf{z}_{i,t} = f_{\theta}([\mathbf{u}_{i,t}; \boldsymbol{\tau}_t]).$$

The backbone is still convolutional and timestamp-level.

What is a time embedding?

T-Rep learns a function

$$h_{\psi} : t \mapsto \tau_t.$$

The embedding should encode useful temporal structure:

- trend,
- periodicity,
- phase,
- distribution shifts,
- position in a local trajectory.

Unlike a fixed Transformer positional encoding, h_{ψ} is trained with the representation model.

Time embeddings as distributions

T-Rep normalizes the time embedding so its components are positive and sum to one:

$$(\tau_t)_k = \frac{\sigma(h_\psi(t))_k}{\sum_j \sigma(h_\psi(t))_j}.$$

Why?

One of the new pretext tasks uses statistical divergence between time embeddings:

$$\text{JSD}(p\|q) = \frac{1}{2}\text{KL}(p\|m) + \frac{1}{2}\text{KL}(q\|m), \quad m = \frac{p+q}{2}.$$

Here JSD is the Jensen-Shannon divergence: a symmetric way to compare two probability vectors.

For many applications, T-Rep uses a Time2Vec-like embedding.

$$h_{\psi}(t) = [\omega_0 t + \phi_0, \sin(\omega_1 t + \phi_1), \dots, \sin(\omega_K t + \phi_K)].$$

- Linear terms can capture trend.
- Periodic terms can capture seasonality or cycles.
- Learning the frequencies adapts the embedding to the dataset.

T-Rep uses four pretext tasks

Task	Source	Main signal
Instance contrast	TS2Vec	separate series
Temporal contrast	TS2Vec	separate timestamps
TE divergence prediction	T-Rep	latent temporal geometry
TE-conditioned forecasting	T-Rep	local predictive context

The new tasks are designed to add continuous time information to the binary contrastive signal.

Notation for T-Rep task heads

Symbol	Meaning
c, c'	two augmented contexts of the same or different series
τ_t	learned time embedding at timestamp t
$\mathcal{G}_1$	scalar regression head for predicting time-embedding divergence
$\mathcal{G}_2$	vector prediction head for predicting a nearby representation
Ω	sampled set of index tuples used in the loss
M	number of sampled tuples in Ω
Δ	sampled time offset for a nearby target timestamp

The heads are trained only for the pretext tasks; downstream tasks use the encoder representations.

Task 1: time-embedding divergence prediction

Take two representation vectors:

$$\mathbf{z}_{i,t}^{(c)}, \quad \mathbf{z}_{j,t'}^{(c')}.$$

Compute their difference and ask a head to predict how different their time embeddings are:

$$\underbrace{\mathcal{G}_1}_{\text{scalar head}} \left(\mathbf{z}_{i,t}^{(c)} - \mathbf{z}_{j,t'}^{(c')} \right) \approx \text{JSD}(\tau_t \parallel \tau_{t'}).$$

This encourages latent distances to reflect a learned notion of temporal difference.

Task 1 loss

$$\mathcal{L}_{div} = \frac{1}{M} \sum_{(i,j,t,t') \in \Omega} \left(\mathcal{G}_1(\mathbf{z}_{i,t} - \mathbf{z}'_{j,t'}) - \text{JSD}(\boldsymbol{\tau}_t \parallel \boldsymbol{\tau}_{t'}) \right)^2.$$

- $\mathcal{G}_1 : \mathbb{R}^K \rightarrow \mathbb{R}$ predicts a scalar divergence.
- Ω contains sampled pairs of instances and timestamps.

Interpretation:

- TS2Vec says “same timestamp or not.”
- T-Rep asks “how different are these learned temporal contexts?”

Task 2: TE-conditioned forecasting

Use a representation at time t and the target time embedding:

$$[\mathbf{z}_{i,t}; \boldsymbol{\tau}_{t+\Delta}].$$

Predict the representation at a nearby timestamp:

$$\underbrace{\mathcal{G}_2}_{\text{vector head}}([\mathbf{z}_{i,t}; \boldsymbol{\tau}_{t+\Delta}]) \approx \mathbf{z}_{i,t+\Delta}.$$

This asks the model to learn representations that know their local temporal neighborhood.

$$\mathcal{L}_{pred} = \frac{1}{MT} \sum_{j \in \Omega_M} \sum_{t \in \Omega_T} \left\| \mathcal{G}_2 \left([\mathbf{z}_{i,t}^{(c_1)}; \boldsymbol{\tau}_{t+\Delta_j}] \right) - \mathbf{z}_{i,t+\Delta_j}^{(c_2)} \right\|^2.$$

$$\Delta_j \sim \mathcal{U}[-\Delta_{\max}, \Delta_{\max}].$$

- $\mathcal{G}_2 : \mathbb{R}^{K+K_\tau} \rightarrow \mathbb{R}^K$ predicts a representation vector.
- Ω_M and Ω_T are sampled instance and timestamp subsets.
- The prediction is local and can be forward or backward in time.

Why predict nearby representations?

- It encourages each timestamp representation to encode surrounding context.
- It makes time embeddings useful for prediction, not only position labels.
- It supports missing-data robustness: nearby context can infer a plausible latent vector.
- A short range avoids turning the whole model into only a forecasting model.

T-Rep is still a representation learner, not just a supervised forecaster.

Full T-Rep loss

Let the four task losses be:

$$\ell_1 = \ell_{inst}, \quad \ell_2 = \ell_{temp}, \quad \ell_3 = \ell_{div}, \quad \ell_4 = \ell_{pred}.$$

T-Rep combines them before the hierarchical loss:

$$\mathcal{L} = \sum_{k=1}^4 \alpha_k \ell_k, \quad \sum_{k=1}^4 \alpha_k = 1.$$

The paper uses equal weights in its reported experiments.

Hierarchical loss remains important

Algorithm 1: Calculating the hierarchical contrastive loss

```
1: procedure HIERLOSS( $r, r'$ )
2:    $\mathcal{L}_{hier} \leftarrow \mathcal{L}_{dual}(r, r')$ ;
3:    $d \leftarrow 1$ ;
4:   while time_length( $r$ ) > 1 do
5:     // The maxpool1d operates along the time axis.
6:      $r \leftarrow \text{maxpool1d}(r, \text{kernel\_size} = 2)$ ;
7:      $r' \leftarrow \text{maxpool1d}(r', \text{kernel\_size} = 2)$ ;
8:      $\mathcal{L}_{hier} \leftarrow \mathcal{L}_{hier} + \mathcal{L}_{dual}(r, r')$ ;
9:      $d \leftarrow d + 1$ ;
10:  end while
11:   $\mathcal{L}_{hier} \leftarrow \mathcal{L}_{hier}/d$ ;
12:  return  $\mathcal{L}_{hier}$ 
13: end procedure
```

T-Rep keeps the TS2Vec hierarchical framework, but the per-scale objective is now a weighted mixture of four pretext losses.

Figure source: Fraikin et al., T-Rep, arXiv:2310.04486; algorithm originally from TS2Vec.

T-Rep is evaluated on point and segment anomalies.

- Yahoo: streaming point anomaly detection.
- Sepsis: segment/timestep abnormality detection in ICU data.
- The learned representation is used by a small downstream anomaly detector or classifier.

Why T-Rep should help: anomalies are often temporal structure violations, not just unusual raw values.

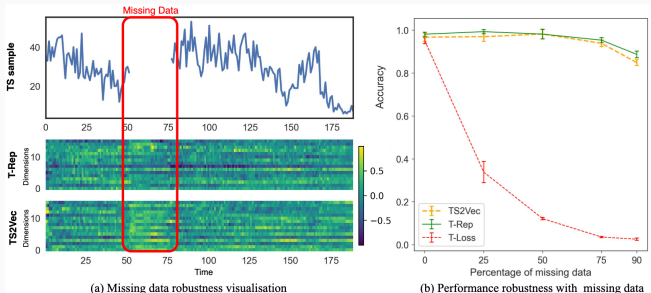
For self-supervised models, the forecasting head is intentionally simple:

$$\hat{\mathbf{X}}_{t+1:t+H} = g_{\phi}(\mathbf{z}_t).$$

In the T-Rep experiments, this is a ridge regression over extracted representations.

The encoder is not trained directly on the final forecasting task, but the pretext tasks encourage predictive temporal information.

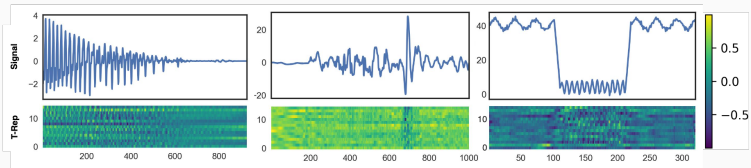
T-Rep missing-data robustness



The paper reports smoother latent representations and stronger accuracy under high missingness.

Figure source: Fraikin et al., T-Rep, arXiv:2310.04486.

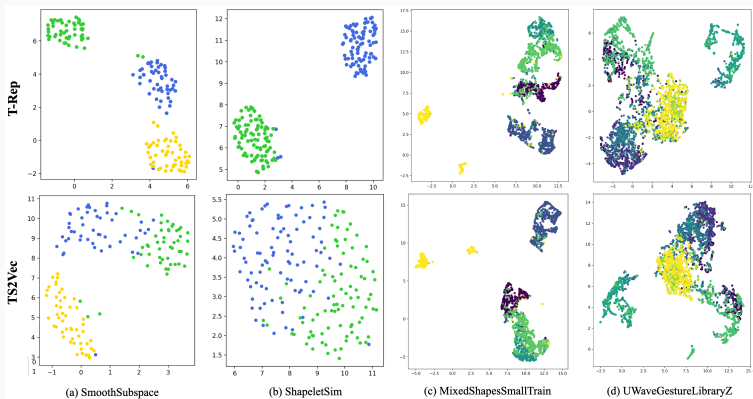
T-Rep latent trajectories



The representation heatmaps show periodicity, spikes, and distribution shifts through time.

Figure source: Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep and TS2Vec clusters



T-Rep keeps strong inter-instance structure while adding more explicit temporal structure.

Figure source: Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep anomaly detection results

Method	Yahoo F1	Sepsis F1
Baseline	0.110	0.241
TS2Vec	0.733	0.619
T-Rep	0.757	0.666

The largest gain is on Sepsis, where missingness and temporal abnormality are central.

Table recreated from Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep classification results

Method	Avg. Acc. on 30 UEA datasets	Avg. Difference
T-Loss	0.657	33.6%
TS2Vec	0.699	2.1%
TNC	0.671	12.3%
TS-TCC	0.667	10.0%
Minirocket	0.707	4.8%
T-Rep	0.706	—

T-Rep is nearly tied with Minirocket while remaining a general representation model.

Table recreated from Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep forecasting results

Method	Avg. Rank		Ranks 1st		Average	
	MSE	MAE	MSE	MAE	MSE	MAE
T-Rep	1.90	1.85	8	8	0.986	0.702
TS2Vec	2.40	2.45	2	1	1.004	0.712
Informer	3.30	3.55	3	3	1.300	0.820
TCN	4.35	4.10	1	1	2.012	1.205
Linear	3.05	3.05	6	7	1.017	0.727

Table recreated from Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep ablation summary

Variant	Forecasting MSE	Sepsis F1
T-Rep	0.986	0.665
w/o TE-conditioned forecasting	1.022	0.392
w/o TE divergence prediction	1.003	0.634
w/o new pretext tasks	0.999	0.513
w/ MLP TE module	1.008	0.443
w/ RBF TE module	1.007	0.401
w/o TE module (= TS2Vec)	1.004	0.610

Table recreated from Fraikin et al., T-Rep, arXiv:2310.04486.

T-Rep takeaways

- T-Rep keeps TS2Vec's strong contextual consistency framework.
- Its main addition is an explicit learned representation of time.
- New pretext tasks make temporal similarity continuous rather than purely binary.
- The gains are especially natural for forecasting, anomaly detection, and missing data.
- The design raises a broader question: how much time structure should be built into the representation objective?

Comparison and Future Directions

TS2Vec vs T-Rep

Dimension	TS2Vec	T-Rep
Main representation	timestamp vectors	timestamp vectors
Backbone	dilated CNN	dilated CNN
Positive pair	same timestamp, two contexts	inherits TS2Vec positives
Main time signal	temporal contrast	contrast + learned time embeddings
New pretext tasks	none beyond dual contrast	TE divergence, TE-conditioned forecasting
Strength	universal multiscale contrast	smoother temporal structure
Best intuition	robust contextual encoder	temporally structured latent trajectory

After TS2Vec and T-Rep: foundation models

Recent work moves from one-dataset encoders to pretrained time-series foundation models.

- **MOMENT**: open time-series foundation models trained on a large Time-series Pile.
- **Chronos**: tokenizes scaled time-series values and trains language-model architectures.
- **TimesFM**: decoder-only patched foundation model for zero-shot forecasting.
- **Moirai**: masked encoder universal forecasting transformer trained on LOTSA.

Sources: MOMENT [arXiv:2402.03885](#); Chronos [arXiv:2403.07815](#); TimesFM [arXiv:2310.10688](#); Moirai [arXiv:2402.02592](#).

New objective families

Direction	Training signal	Typical use
Masked reconstruction	recover hidden patches	representation, imputation
Tokenized forecasting	next token likelihood	zero-shot forecasts
Patched decoding	autoregressive patches	long horizons
Universal masking	flexible patch/variant setup	cross-domain forecasting

The trend is not only “bigger models”; it is also a search for objectives that survive across domains, scales, and sampling rates.

- **UniTS** aims to unify forecasting, classification, anomaly detection, and imputation with task tokenization.
- **Time-MoE** scales decoder-only time-series models with sparse mixture-of-experts layers.
- The frontier is shifting toward zero-shot and few-shot transfer.
- Specialized SSL encoders still matter when interpretability, timestamp granularity, or domain control is important.

Sources: UniTS [arXiv:2403.00131](#); Time-MoE [arXiv:2409.16040](#).

Open questions

- How do we avoid benchmark contamination in large pretraining corpora?
- How should models represent irregular sampling and heterogeneous frequencies?
- How should covariates, metadata, and event calendars enter the model?
- Can zero-shot foundation models be calibrated enough for high-stakes decisions?
- When do fine-grained SSL representations beat large forecasting foundation models?

Practical checklist

- Start from the downstream granularity: instance, subsequence, or timestamp.
- Audit which invariances are safe for the domain.
- Use chronological validation if deployment is chronological.
- Compare against simple classical and supervised baselines.
- Test missingness and distribution shift deliberately.
- Inspect representations, not only metrics.

Final takeaways

- Time-series SSL is mostly about designing the right pretext signal for temporal structure.
- TS2Vec shows that contextual consistency and hierarchy produce strong general timestamp representations.
- T-Rep shows that explicit learned time embeddings can make the latent trajectory smoother and more useful.
- Newer foundation models scale pretraining, but they do not remove the need to understand objectives, data splits, and task granularity.